



BRAINWARE UNIVERSITY

BRAINWARE UNIVERSITY

PYTHON PROJECT-BASED LEARNING REPORT (WEEK 3)

1. Basic Information

Title of the Project:	Console-Based Management Systems (Programs 1- 3)
Programs Covered:	Program 1: Bank Management System Program 2: Hotel Booking System Program 3: Library Management System
Name(s) of Student(s):	Soumyadwip Das (BWU/BTS/25/169) Ankita Paul (BWU/BTS/25/180) Sonia Naskar (BWU/BTS/25/195) Joyeta Mondal (BWU/BTS/25/214) Sayantan Bharati (BWU/BTS/25/503)
Programme & Semester:	B.Tech CSE & 3rd Semester
Course Name & Code:	Python Programming Lab (BES09013)
Name of the Guide/Supervisor:	MR. PRANAB GHARAI & MR. INDRANIL DAS



2. Introduction and Background

Week 3 focused on three menu-driven Python applications: Bank Management System, Hotel Booking System, and Library Management System. All use in-memory data structures, a common boxed terminal interface, and menu-driven control flow. This report presents each program separately, followed by a combined timeline, tools, and references

3. Program 1: Bank Management System

3.1 Introduction and Background

A Bank Management System simplifies banking operations such as account creation, balance enquiry, deposits, and withdrawals. This project develops a console-based Python application using in-memory data structures, featuring unique account generation, PIN authentication, and a boxed terminal interface.

3.2 Objectives

- Develop a menu-driven bank management system.
- Generate unique account numbers.
- Implement PIN-based authentication.
- Support deposit, withdrawal, and balance enquiry.
- Build a consistent terminal interface.

3.3 Problem Statement / Driving Question

How can basic banking operations be implemented securely in a menu-driven console application using Python without an external database?

3.4 Methodology

- **Program Design:** Created a boxed terminal interface using reusable helper functions and an ASCII-art banner.
- **Data Structures:** Used a USERS dictionary to store account details and an ACCOUNTS list to maintain unique account numbers.
- **Core Banking Functions:** Implemented account generation, account creation, authentication, deposit, and withdrawal with balance validation.
- **Menu-Driven Control Flow:** Built a continuous menu with Create Account, Check Balance, Withdraw, Deposit, and Exit options, along with input validation and error handling.

3.5 Expected Outcomes / Deliverables

- A functional bank management system.
- Unique account number generation.
- Secure PIN-based authentication.
- Input validation and error handling.
- A user-friendly boxed terminal interface



3.6 Core Logic

```
def generate_account():
    while True:
        num = math.floor(random.random() * (10 ** 10))
        account_no = "A" + str(num)
        if account_no not in ACCOUNTS:
            return account_no # ensures every account number is unique

def withdraw(account, amount):
    if amount <= USERS[account]['balance']:
        USERS[account]['balance'] -= amount
        return True # withdrawal allowed, balance updated
    return False # insufficient funds, transaction rejected
```

The generate_account() function creates a unique account number by generating a random 10-digit value and ensuring it does not already exist. The withdraw() function allows transactions only when sufficient balance is available, preventing negative account balances.

3.7 Output Screenshot



4. Program 2: Hotel Booking System

4.1 Introduction and Background

A Hotel Booking System simplifies room reservation by managing room availability, bookings, and check-outs. This project develops a console-based Python application using in-memory data structures with a consistent boxed terminal interface.

4.2 Objectives

- Develop a menu-driven hotel booking system.
- Manage multiple room categories and availability.
- Generate unique booking IDs.
- Calculate booking cost based on room type and stay duration.
- Support booking, check-out, and booking history.
- Build a consistent terminal interface.

4.3 Problem Statement / Driving Question

How can a hotel manage room availability, bookings, check-outs, and billing using a simple console-based application without an external database?

4.4 Methodology

- **Program Design:** Created a boxed terminal interface using reusable helper functions and an ASCII-art banner.
- **Data Structures:** Used ROOM_TYPES to store room details and BOOKINGS to manage reservation records.
- **Core Booking Functions:** Implemented room availability checking, booking creation with unique booking IDs, total cost calculation, and check-out functionality.
- **Menu-Driven Control Flow:** Built a continuous menu with room viewing, booking, check-out, and booking history options, including input validation and error handling.

4.5 Expected Outcomes / Deliverables

- A functional hotel booking system.
- Real-time room availability tracking.
- Unique booking ID generation.
- Accurate booking cost calculation.
- Input validation and error handling.
- A user-friendly boxed terminal interface.



4.6 Core Logic

```
def available_rooms(room_type):
    booked = sum(
        1 for b in BOOKINGS.values()
        if b["room_type"] == room_type
        and b["status"] == "Booked"
    )
    # vacancy = total rooms - active bookings
    return ROOM_TYPES[room_type]["count"] - booked

def book_room(name, room_type, nights):
    if available_rooms(room_type) <= 0:
        return None, f"No {room_type} rooms available."
    booking_id = 1001 + len(BOOKINGS) # unique, sequential
    total = ROOM_TYPES[room_type]["price"] * nights
    BOOKINGS[booking_id] = {
        "name": name, "room_type": room_type,
        "nights": nights, "total": total, "status": "Booked"
    }
    return booking_id, total
```

The `available_rooms()` function calculates the number of vacant rooms by subtracting active bookings from the total room count. The `book_room()` function checks availability, generates a unique booking ID, calculates the total cost, and stores the reservation, ensuring rooms cannot be overbooked.

4.7 Output Screenshot

```
HOTEL
BOOKING
SYSTEM

Terminal Hotel Booking System

[1] View Rooms & Availability
[2] Book a Room
[3] Check-Out / Free a Room
[4] View All Bookings
[5] Exit

> Enter option: 2
> Enter guest name: ram

SELECT ROOM TYPE:

Single - Rs.1000/night
Double - Rs.2000/night
Suite - Rs.5000/night

> Enter room type: double
> Enter number of nights: 5

ROOM BOOKED:

Booking ID : 1001
Guest      : ram
Room type  : double
Nights     : 5
Total amount: Rs.10000
```



5. Program 3: Library Management System

5.1 Introduction and Background

A Library Management System helps manage book records, availability, and borrowing activities. This project develops a console-based Python application that supports adding, viewing, issuing, returning, and searching books using in-memory data structures and a boxed terminal interface.

5.2 Objectives

- Develop a menu-driven library management system.
- Add and manage book records with unique IDs.
- Support book issue and return operations.
- Search books by ID, title, or author.
- Display book records and availability.
- Prevent invalid issue and return operations.

5.3 Problem Statement / Driving Question

How can a console-based application efficiently manage library records while preventing invalid book issue and return operations?

5.4 Methodology

- **Program Design:** Created a boxed terminal interface with reusable helper functions and an ASCII-art banner.
- **Book Record Management:** Stored book details in a `BOOKS` dictionary, implemented duplicate ID validation, and displayed records in a formatted table.
- **Issue, Return & Search Logic:** Developed functions to issue, return, and search books with proper validation and tested all menu operations.

5.5 Expected Outcomes / Deliverables

- A functional library management system.
- Accurate book issue and return tracking.
- Search functionality for book records.
- A clear, tabular catalogue display.
- Scope for future enhancements such as due dates and persistent storage.



5.6 Core Logic

```
def issue_book(book_id, member):
    # Only an existing, currently-Available book can be issued
    if book_id in BOOKS and BOOKS[book_id]["status"] == "Available":
        BOOKS[book_id]["status"] = "Issued"
        BOOKS[book_id]["issued_to"] = member
        return True
    return False

def return_book(book_id):
    # Only a book currently marked Issued can be returned
    if book_id in BOOKS and BOOKS[book_id]["status"] == "Issued":
        BOOKS[book_id]["status"] = "Available"
        BOOKS[book_id]["issued_to"] = ""
        return True
    return False
```

The `issue_book()` and `return_book()` functions update a book's status only when the requested action is valid, ensuring a book cannot be issued twice or returned before being issued.

5.7 Output Screenshot

```
LIBRARY MANAGEMENT SYSTEM
Terminal Library Management System

[1] Add Book
[2] View All Books
[3] Issue Book
[4] Return Book
[5] Search Book
[6] Exit

> Enter option: 1
> Enter book ID: 002
> Enter book title: book1
> Enter author name: auth1

BOOK ADDED:
Book ID : 002
Title   : book1
Author  : auth1
Status  : Available

[1] Add Book
[2] View All Books
[3] Issue Book
[4] Return Book
[5] Search Book
[6] Exit

> Enter option: |
```



6. Tools & Technologies Used

Tool / Technology	Purpose
Python 3.x	Core programming language
Dictionary & List	Store accounts, bookings, and book records
random & math Modules	Generate unique account numbers
f-Strings	Format console output
Command-Line Interface	Run and test the applications
Text Editor / IDE	Develop and debug the programs

7. References

- Python Official Documentation
- Python Dictionaries Documentation
- Python random & math Module Documentation
- Python String Formatting Documentation
- GeeksforGeeks – Python Management System Tutorials
- Brainware University – Python Programming Lab Course Guidelines



BRAINWARE UNIVERSITY

8. Signatures

Signature of Student(s):

Date of Submission:

Signature of Guide/Supervisor:
